AUDITABLE AI AGENTS: AN IMMUTABLE ON-CHAIN AUDIT TRAIL FOR LLM AGENT ACTIONS

PHASE I REPORT

Submitted by

MANIKANDAN SANKARANARAYANAN

(Register No. [REGISTER NUMBER])

in partial fulfillment for the award of the degree of

MASTER OF ENGINEERING IN COMPUTER SCIENCE AND ENGINEERING

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
S VEERSAMY CHETTIYA COLLEGE OF ENGINEERING
(Affiliated to Anna University, Chennai)

[MONTH YEAR]

S VEERSAMY CHETTIYA COLLEGE OF ENGINEERING

(Affiliated to Anna University, Chennai)

BONAFIDE CERTIFICATE

Certified that this Report titled "AUDITABLE AI AGENTS: AN IMMUTABLE ON-CHAIN AUDIT TRAIL FOR LLM AGENT ACTIONS" is the bonafide work of MANIKANDAN SANKARANARAYANAN (Register No. [REGISTER NUMBER]) who carried out the work under my supervision. Certified further that to the best of my knowledge the work reported herein does not form part of any other thesis or dissertation on the basis of which a degree or award was conferred on an earlier occasion on this or any other candidate.

<<Signature of the HOD with date>>
[HOD NAME]
Professor and Head
Department of Computer Science and Engineering
S Veersamy Chettiya College of Engineering

<<Signature of the Supervisor with date>>
[SUPERVISOR NAME]
[ACADEMIC DESIGNATION]
Department of Computer Science and Engineering
S Veersamy Chettiya College of Engineering

ABSTRACT

Autonomous agents built on large language models (LLMs) call tools and act on behalf of users, yet their records are usually ordinary log files that anyone with server access can edit or delete without leaving evidence. This work presents a tamper-evident audit trail for LLM agent actions. Each step of an agent run, namely the task, model response, tool call and tool result, is written to a hash-chained log. Batches of entries are folded into Merkle roots anchored in a smart contract, so only 32-byte hashes, never raw logs, are stored on the blockchain. A verifier recomputes every hash from the stored log, compares each batch root with the anchored value, and reports the first altered entry or batch.

The prototype, implemented in Python and Solidity on a local Ethereum development chain, was exercised with a scripted model, a local open-weight model and an adapter for a hosted model. In randomised tests, seven classes of tampering were each applied to 300 logs and all 2,100 attacks were detected, with no false alarms on 300 untouched logs. A consistently rewritten log, which defeats a hash chain alone, was caught by the on-chain roots. Python-generated Merkle proofs verified inside the contract for trees of up to 100 entries, and anchoring cost 79,518 to 93,100 gas per batch in a first run. The trail proves a log was not altered after anchoring, not that the agent's outputs were correct. Phase II will measure gas against batch size, overhead and scaling.

ABSTRACT (TAMIL)

பெரிய மொழி மாதிரிகளால் (LLM) இயங்கும் தன்னியக்க முகவர்கள் கருவிகளை அழைத்துப் பயனர்களின் சார்பாகச் செயல்படுகின்றன. ஆனால் அவற்றின் பதிவுகள் பொதுவாகச் சாதாரண பதிவுக் கோப்புகளாகவே உள்ளன; சேவையகத்தை அணுகக்கூடிய எவரும் எந்தத் தடயமும் இல்லாமல் அவற்றை மாற்றவோ நீக்கவோ முடியும். இந்த ஆய்வு LLM முகவர்களின் செயல்களுக்கான சேதத்தைக் கண்டறியக்கூடிய (tamper-evident) தணிக்கைப் பாதையை முன்வைக்கிறது. முகவரின் ஒவ்வொரு படியும், அதாவது பணி, மாதிரியின் பதில், கருவி அழைப்பு, கருவியின் முடிவு ஆகியவை, ஹேஷ்-சங்கிலியால் இணைக்கப்பட்ட பதிவில் எழுதப்படுகின்றன. பதிவு உள்ளீடுகளின் தொகுதிகள் மெர்க்கிள் மூலமாக (Merkle root) மாற்றப்பட்டு ஸ்மார்ட் ஒப்பந்தத்தில் நங்கூரமிடப்படுகின்றன; இதனால் 32-பைட் ஹேஷ் மதிப்புகள் மட்டுமே பிளாக்செயினில் சேமிக்கப்படுகின்றன, மூலப் பதிவுகள் அல்ல.

சரிபார்ப்பு நிரல் சேமிக்கப்பட்ட பதிவிலிருந்து ஹேஷ்களை மீண்டும் கணக்கிட்டு, நங்கூரமிடப்பட்ட மதிப்புகளுடன் ஒப்பிட்டு, முதலில் மாற்றப்பட்ட உள்ளீட்டை அல்லது தொகுதியைக் கண்டறிகிறது. பைத்தான் மற்றும் சாலிடிட்டியில் உருவாக்கப்பட்ட இந்த முன்மாதிரியில், ஏழு வகையான சேதங்கள் ஒவ்வொன்றும் 300 பதிவுகளில் செய்யப்பட்டபோது, 2,100 தாக்குதல்களும் கண்டறியப்பட்டன; மாற்றப்படாத 300 பதிவுகளில் தவறான எச்சரிக்கை எதுவும் இல்லை. இந்தத் தடம் பதிவு நங்கூரமிடப்பட்ட பின் மாற்றப்படவில்லை என்பதை மட்டுமே நிரூபிக்கும்; முகவரின் வெளியீடுகள் சரியானவை என்பதை அல்ல.

ACKNOWLEDGEMENT

I express my sincere gratitude to [SUPERVISOR NAME], [ACADEMIC DESIGNATION], Department of Computer Science and Engineering, for the guidance, encouragement and constructive criticism that shaped this work. I thank [HOD NAME], Professor and Head of the Department, and the Principal of S Veersamy Chettiya College of Engineering for providing the facilities needed to carry out this project. I am grateful to the faculty members and the project review committee for their suggestions, and to my family and friends for their support.

MANIKANDAN SANKARANARAYANAN

TABLE OF CONTENTS

ABSTRACT **iii**
ABSTRACT (TAMIL) **iv**
LIST OF TABLES **ix**
LIST OF FIGURES **x**
LIST OF ABBREVIATIONS **xi**
CHAPTER 1 INTRODUCTION **1**
1.1 General 1
1.2 Problem statement 1
1.3 Motivation 1
1.4 Objectives 1
1.5 Scope and limitations 2
1.6 Work completed in Phase I 2
1.7 Organisation of the report 2
CHAPTER 2 REVIEW OF LITERATURE **3**
2.1 Introduction 3
2.2 Tamper-evident logging 3
2.3 Merkle trees and transparency logs 3
2.4 Blockchains and smart contracts 4
2.5 LLM agents and tool use 4
2.6 AI accountability 4
2.7 Summary and research gap 4
CHAPTER 3 THEORETICAL BACKGROUND, METHODOLOGY AND DESIGN **6**
3.1 Introduction 6
3.2 Threat model 6
3.3 Cryptographic building blocks 6
3.3.1 Hash function 6
3.3.2 Canonical serialisation 7
3.4 Hash-chained log 7
3.5 Merkle batches and inclusion proofs 7
3.6 On-chain anchoring contract 8
3.7 System architecture 8

3.8 Verification algorithm

9
3.9 Design decisions 9
CHAPTER 4 IMPLEMENTATION 11
4.1 Environment and tools 11
4.2 Module structure 11
4.3 The agent and its tools 12
4.4 Model adapters 12
4.5 The smart contract 12
4.6 The anchoring pipeline 12
4.7 The verifier 13
4.8 Demonstration run 13
CHAPTER 5 TESTING AND RESULTS TO DATE 14
5.1 Test strategy 14
5.2 Test suite 14
5.3 Detection of tampering 14
5.4 What each layer catches 15
5.5 Agreement between Python and Solidity 15
5.6 Run with a real local model 15
5.7 Discussion 15
CHAPTER 6 CONCLUSIONS AND WORK SCHEDULE FOR PHASE II 17
6.1 Conclusions 17
6.2 Work schedule for Phase II 17
6.3 Risks 17
APPENDIX 1 SMART CONTRACT SOURCE 19
APPENDIX 2 MERKLE TREE MODULE 21
APPENDIX 3 HASH-CHAIN MODULE 23
APPENDIX 4 DEMONSTRATION OUTPUT 25
APPENDIX 5 LIST OF PUBLICATIONS 26
REFERENCES 27

LIST OF TABLES

Table 2.1 Related approaches and what they leave open **4**
Table 3.1 Threat model **6**
Table 3.2 Main design decisions and reasons **9**
Table 4.1 Software environment **11**
Table 4.2 Source modules **11**
Table 4.3 Gas used by the anchoring transactions in the demonstration run **13**
Table 5.1 Test suite summary **14**
Table 5.2 Detection results over random logs **14**
Table 6.1 Phase II work plan **17**

LIST OF FIGURES

Figure 3.2 Hash-chained log entries 7
Figure 3.3 Merkle tree and inclusion proof for one entry 7
Figure 3.1 System architecture 8
Figure 5.1 Detection by the hash chain alone and with on-chain roots 15

LIST OF ABBREVIATIONS

Abbreviation	Expansion
ABI	Application Binary Interface
AI	Artificial Intelligence
API	Application Programming Interface
EVM	Ethereum Virtual Machine
FIPS	Federal Information Processing Standard
JSONL	JSON Lines
LLM	Large Language Model
NIST	National Institute of Standards and Technology
RPC	Remote Procedure Call
SHA	Secure Hash Algorithm

CHAPTER 1
INTRODUCTION

1.1 General

Large language models have moved from answering questions to taking actions. An LLM agent receives a task, decides which tool to use, such as a search engine, a database or a file editor, observes the result, and repeats until it can give a final answer. Such agents are being deployed to read documents, call services and make changes on behalf of users. The more authority an agent has, the more important it becomes to know afterwards exactly what it did.

Today the record of an agent's behaviour is usually an ordinary log file or a database table. These records have a weakness that is easy to overlook: whoever controls the machine that stores them can edit or delete entries, and nobody else can prove that the record was left untouched. An audit trail that its subject can quietly rewrite is not much of an audit trail.

1.2 Problem statement

Given an LLM agent that runs on infrastructure controlled by one party, how can a second party later verify that the record of the agent's actions is complete and unchanged, without having to trust the first party, and at a cost small enough to be practical?

1.3 Motivation

Three observations motivate the work. First, agent actions can have real consequences, from wrong answers to unauthorised operations, so disputes about what happened are likely. Second, techniques for tamper-evident logging exist but were designed for ordinary software logs, not for the specific structure of agent traces (prompt, model response, tool call, tool result). Third, public blockchains offer a witness that no single party controls, but storing whole logs on a blockchain is costly and exposes data, so the design must keep almost everything off-chain.

1.4 Objectives

1. Design a log format for agent actions in which any edit, deletion, insertion or reordering is detectable.
2. Anchor compact commitments to the log on a blockchain so that a consistently rewritten log is also detectable.
3. Build a verifier that reports whether a stored log is intact and, if not, where it was altered.
4. Show by automated test that every class of tampering considered is detected and that untouched logs are never flagged.
5. Demonstrate the system with a real language model that runs locally, and measure the cost of anchoring.

1.5 Scope and limitations

The work covers integrity of the record after anchoring. It does not establish that an agent's output was correct or truthful. It does not prevent a compromised logger from writing false entries before they are anchored. The prototype runs on a local development chain and is not a production system. These limits are stated again in Chapters 3 and 5 because they determine what the results can be taken to mean.

1.6 Work completed in Phase I

Phase I produced a working pipeline (agent, audit log, Merkle batching, smart contract, verifier), a test suite of 79 Python tests and 18 contract tests, a demonstration with a locally hosted language model, and a first measurement of anchoring cost. Phase II will carry out the planned measurements (Chapter 6).

1.7 Organisation of the report

Chapter 2 reviews the literature. Chapter 3 gives the threat model, the cryptographic background and the design. Chapter 4 describes the implementation. Chapter 5 reports testing and results to date. Chapter 6 gives conclusions and the work schedule for Phase II. The appendices list the main source files and the output of a demonstration run.

CHAPTER 2
REVIEW OF LITERATURE

2.1 Introduction

The review covers five areas: tamper-evident logging, Merkle trees and transparency logs, blockchains and smart contracts, LLM agents, and AI accountability. Items marked [VERIFY] in the reference list were written from memory and must be checked against the source before submission.

2.2 Tamper-evident logging

Schneier and Kelsey (1999) described secure audit logs in which each record is cryptographically linked to the ones before it, so that an attacker who takes over a machine cannot undetectably alter records written earlier. Their work established the idea of chaining log entries, and also its limit: a chain held entirely by the party being audited can be recomputed from the point of tampering onward.

Crosby and Wallach (2009) considered an untrusted logger that serves a number of clients who store events in the log, with auditors who keep the logger honest. They proposed data structures that let clients and auditors check, efficiently and with short proofs, that the log has not been tampered with. Their threat model, an untrusted logger checked by outsiders who hold commitments, is the closest to the one in this project, and their paper is used as the base paper.

2.3 Merkle trees and transparency logs

A Merkle tree (Merkle 1988) hashes data blocks into leaves and hashes pairs of nodes upward to a single root, so that one short value commits to a large collection and a logarithmic-size proof shows that one block belongs to it. Haber and Stornetta (1991) showed how linking documents by hash can prove when a document existed.

Certificate Transparency applies these ideas at internet scale. RFC 9162 (Laurie et al. 2021) specifies a protocol for publicly logging TLS

certificates in append-only Merkle-tree logs so that anyone can audit the issuance of certificates. It replaces the earlier RFC 6962 (Laurie et al. 2013), which is now obsolete. The log operators publish signed commitments; this project instead publishes commitments through a blockchain, which removes the need to trust a signing operator.

2.4 Blockchains and smart contracts

Nakamoto (2008) described a peer-to-peer system in which transactions are ordered by a chain of hashed blocks, producing a record that no single party can change. Wood (Yellow Paper) specified Ethereum as a generalised transaction ledger in which programs, called smart contracts, run on every node and keep state on the chain. Each operation consumes gas, so storing data on-chain is expensive. This is why the design in this report stores one 32-byte root per batch and keeps the log itself off-chain.

2.5 LLM agents and tool use

Yao et al. (2023) proposed ReAct, in which a model alternates between reasoning and actions that query external tools, reducing made-up answers and making the behaviour inspectable. Anthropic (2024) describes practical agent designs and recommends starting with the simplest system that works. The agent loop used in this project, in which the model requests a tool, the program runs it and the result is returned to the model, follows this pattern. An agent trace therefore consists of the task, the model's responses, the tool calls and the tool results, and those are the events recorded.

2.6 AI accountability

The NIST AI Risk Management Framework (2023) is a voluntary framework for managing the risks of AI systems and building trustworthiness into their design, development, use and evaluation. Traceable records of system behaviour support the accountability that the framework calls for. This project contributes one mechanism for such records.

2.7 Summary and research gap

Table 2.1 Related approaches and what they leave open

Approach	**What it provides**	**What it leaves open for agent audit**
Hash-chained logs (Schneier and Kelsey 1999)	Detects edits, deletions and reordering	Cannot detect a chain recomputed by the log owner
Merkle-tree logs (Crosby and Wallach 2009)	Short proofs; untrusted logger model	Relies on auditors holding commitments; not designed for agent traces
Transparency logs (RFC 9162)	Public, append-only, auditable	Needs trusted signing operators; designed for certificates
Blockchain ledgers (Nakamoto 2008; Wood)	Witness no single party controls	Costly for whole logs; no agent-specific format

The gap addressed here is the combination: an agent-specific log format, Merkle batching with a small on-chain footprint, tests showing which attacks each layer catches, and a measured cost.

CHAPTER 3
THEORETICAL BACKGROUND, METHODOLOGY AND DESIGN

3.1 Introduction

This chapter states the threat model, explains the cryptographic building blocks, and describes the design of the audit trail.

3.2 Threat model

The attacker has access to the stored log and may edit, delete, insert, reorder or truncate entries and may recompute any hash. The attacker cannot change a value that has already been written to the blockchain. The honest logger is assumed to record events correctly at the time they happen.

Table 3.1 Threat model

Actor	**Can**	**Cannot**
Dishonest operator or attacker with file access	Alter the stored log; recompute hashes	Change a root already on the blockchain
Blockchain	Keep roots unchanged and timestamped	Know whether an entry was true
Out of scope	A logger that writes false entries before anchoring; a stolen agent key; denial of service; attacks on the chain itself	

The consequence is that the system provides tamper evidence after anchoring. It does not provide tamper prevention, and it does not provide truthfulness.

3.3 Cryptographic building blocks

3.3.1 Hash function

A cryptographic hash function maps any input to a fixed-size digest. The same input always gives the same digest, a small change in the input

changes the digest completely, and it is infeasible to find two inputs with the same digest. This project uses SHA-256, which produces 32 bytes and is defined in FIPS 180-4 (NIST 2015). SHA-256 is available in the Python standard library and as a built-in function in Solidity, so the verifier and the contract compute identical values.

3.3.2 Canonical serialisation

The same data can be written as text in many ways, which would produce different hashes. Each entry is therefore serialised as canonical JSON, with keys sorted, no whitespace and UTF-8 encoding, before it is hashed. Two programs on different machines then obtain identical bytes for identical data.

3.4 Hash-chained log

Each log entry holds an index, a timestamp, a payload, the hash of the previous entry (prev_hash) and its own hash (entry_hash), computed over the other four fields. The first entry points at an all-zero value. Figure 3.2 shows the structure.

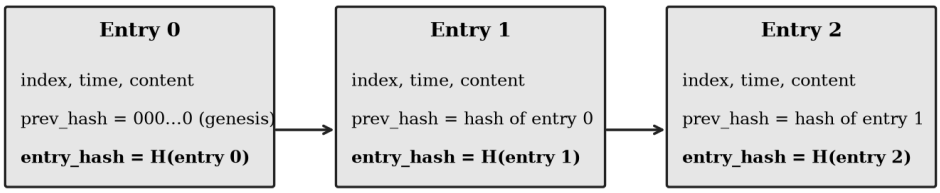


Figure 3.2 Hash-chained log entries

Editing an entry changes its hash and breaks the link from the next entry. Deleting, inserting or reordering entries breaks the index sequence and the links. An attacker can still edit entry k and recompute the hash of every later entry. The resulting chain is internally consistent, so a chain on its own cannot detect this attack. The anchoring layer exists for that reason.

3.5 Merkle batches and inclusion proofs

Every B entries, their hashes are combined into a Merkle tree and the root is anchored. Figure 3.3 shows a tree of four entries and the proof for entry 2.

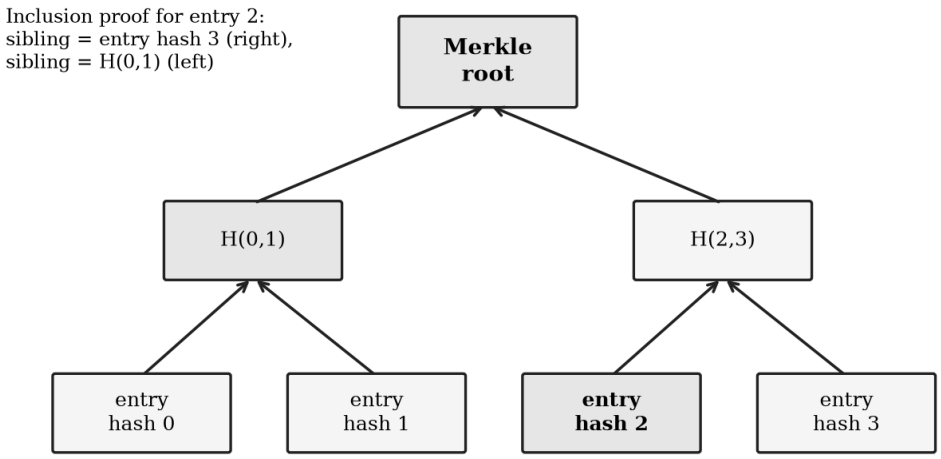


Figure 3.3 Merkle tree and inclusion proof for one entry

Two details prevent known weaknesses. First, leaves are hashed as the SHA-256 of the byte 0x00 followed by the entry hash, and inner nodes as the SHA-256 of 0x01 followed by the two child hashes. This domain separation stops an inner node from being passed off as a leaf. Second, when a level has an odd number of nodes, the unpaired node is carried up unchanged instead of being duplicated, because duplicating the last leaf lets two different logs produce the same root.

3.6 On-chain anchoring contract

The smart contract AuditAnchor stores, for each sending account, a list of batches. A batch records the Merkle root, the index of its first entry, the number of entries and the block time. Batches must be contiguous: the first index of a new batch equals the first index plus the count of the previous one. The contract has no function that edits or deletes a batch. It also offers a read-only function that re-computes a Merkle path on-chain and returns whether an entry belongs to a stored batch.

3.7 System architecture

Figure 3.1 shows the data flow. The agent writes each step to the audit log, which is saved off-chain. Every B entries a root is computed and anchored. The verifier loads the log file and reads the roots from the contract.

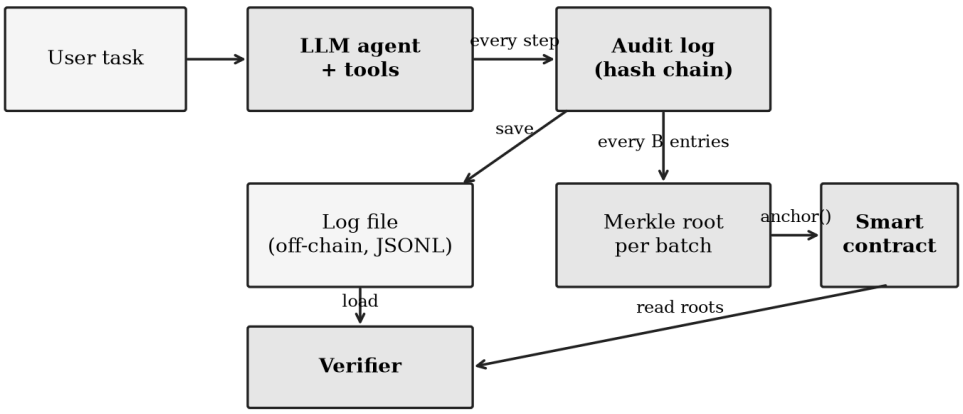


Figure 3.1 System architecture

The code is organised in three layers. The core and domain layers contain only pure functions and immutable data, with no file, network or console access. The application layer contains everything with side effects: reading and writing files, talking to the blockchain and to language models. This separation keeps the cryptographic logic easy to test.

3.8 Verification algorithm

The verifier takes the stored entries and the list of anchored batches. It first checks the hash chain: indices must run 0, 1, 2 and so on; each prev_hash must equal the previous entry's hash; each entry_hash must equal the hash recomputed from the entry. It then recomputes the Merkle root of each batch from the stored entry hashes and compares it with the root on-chain. A batch whose entries extend past the end of the stored log indicates truncation. Entries after the last anchored batch are reported as not yet anchored, not as errors.

If the chain check fails, the first failing entry is reported exactly. If the chain is consistent but a root differs, only the batch can be reported, since the attacker may have rewritten everything from some point on.

3.9 Design decisions

Table 3.2 Main design decisions and reasons

Decision	**Choice**	**Reason**
Hash function	SHA-256	Identical in Python and Solidity
What goes on-chain	One Merkle root per batch	Low gas, no data exposure

Decision	**Choice**	**Reason**
Merkle prefixes	0x00 leaf, 0x01 node	Prevents leaf and node confusion
Odd node	Carried up	No duplicate-leaf collisions
Chain for development	Local Hardhat network	Free, fast, repeatable
Model interface	One function, llm(messages) returns action	Audit layer is independent of the model
Code style	Pure functions, frozen data	Simple to test and reason about

CHAPTER 4
IMPLEMENTATION

4.1 Environment and tools

Table 4.1 Software environment

Component	Version
Python	3.14.6
Node.js	26.5.0
Hardhat	2.29.1
Solidity compiler	0.8.28
web3.py	8.0.0
Anthropic Python SDK	2.3.0
pytest	9.2.0
Ollama	0.40.13
Local model	qwen3:8b (about 5.2 GB)
Computer	Apple M5 Pro, 24 GB memory

4.2 Module structure

Table 4.2 Source modules

Module	Role
src/core/hashing.py	Canonical JSON and SHA-256
src/domain/chain.py	Log entries, hash chain, chain verification
src/domain/merkle.py	Merkle root, inclusion proofs
src/domain/audit.py	Batch planning and verification against anchored batches
src/domain/agent.py	Tools, scripted model, the agent loop
src/app/logstore.py	Save and load the log as JSON Lines
src/app/chain_client.py	Start the local chain, deploy the contract, anchor batches, read batches
src/app/ollama_llm.py	Adapter for local models served by Ollama
src/app/anthropic_llm.py	Adapter for the Anthropic API
src/app/run_demo.py	End-to-end demonstration

Module	Role
contracts/AuditAnchor.sol	The smart contract

4.3 The agent and its tools

The demonstration agent has three tools: a calculator that evaluates arithmetic safely by walking the parsed expression tree and rejecting anything that is not arithmetic, a word counter, and a capital-city lookup. The agent loop asks the model for an action. If the action is a final answer the loop ends; if it is a tool call the program runs the tool and appends the result to the conversation. A step limit prevents endless loops. A failing tool does not crash the run; the error is recorded as an event. Every step appends an entry to the log, and the log is a tuple that is passed along and returned, never modified in place.

4.4 Model adapters

Every model is wrapped as a function from the message list to an action. Three implementations exist. A scripted model decides its tool calls from the task text and is used for fast, deterministic tests. An adapter for Ollama calls a local server over HTTP using only the Python standard library, with temperature 0, a fixed random seed and reasoning mode off; it was run with qwen3:8b. An adapter for the Anthropic API replays the model's output blocks unchanged on later turns, as the API requires; it is tested against a stand-in client but has not been run against the live service because no API key was available in the development environment. Each logged model response carries the model name, token counts and decoding options.

4.5 The smart contract

The contract is written in Solidity 0.8.28 (Appendix 1). Its anchor function rejects empty batches, computes the first index from the previous batch, appends the batch and emits an event. Reading functions are free to call. The inclusion function mirrors the Python Merkle code byte for byte.

4.6 The anchoring pipeline

The anchoring function asks the contract how many entries are already covered, plans new batches from that point, builds each batch root with the same Merkle code the verifier uses, and sends one transaction per batch.

Because the state is read from the chain each time, running the function twice never anchors a batch twice. A final partial batch is anchored only when requested.

4.7 The verifier

The verifier implements the algorithm of Section 3.8. A separate command-line script checks only the hash chain of a log file, without needing a blockchain, so that a log can be examined quickly. It exits with code 0 for an intact log, 1 for tampering and 2 for a usage error.

4.8 Demonstration run

The demonstration starts a local chain, runs the agent with the local model, saves the log, anchors it with a batch size of five, and then verifies the log against the chain after each of several attacks. The agent completed the task (12 audit entries, three tool calls) and the entries were anchored in three batches.

Table 4.3 Gas used by the anchoring transactions in the demonstration run

Batch	Entries	Gas used
0	0 to 4	93,100
1	5 to 9	79,518
2	10 to 11	79,518

The first batch costs more because it creates the first storage slots for the sending account. The full console output is in Appendix 4.

CHAPTER 5
TESTING AND RESULTS TO DATE

5.1 Test strategy

Testing has three aims: to show that every kind of tampering is detected, to show that untouched logs are never flagged, and to show that the Python code and the contract agree on hashing. Randomised tests use fixed seeds so that any failure can be reproduced.

5.2 Test suite

Table 5.1 Test suite summary

Suite	Tests	Focus
Python, core and domain	79 in total	Hashing, chain, Merkle, tools, log store, adapters, command line, live checks
Contract (Hardhat)	18	Storage, events, reverts, isolation, gas ceiling, agreement with Python proofs

The live tests need a running service. Three of them ran against a real local chain or the real local model.

5.3 Detection of tampering

For each of seven attack types, 300 random logs of 2 to 40 entries were generated, anchored with a random batch size, attacked once, and verified.

Table 5.2 Detection results over random logs

Attack	Trials	Detected
Edit an entry's content	300	300
Edit an entry's timestamp	300	300
Edit a stored hash	300	300
Delete an entry	300	300
Insert a duplicate entry	300	300
Swap two entries	300	300
Truncate the log	300	300

Attack	Trials	Detected
Untouched logs (false alarms)	300	0 flagged

An edit was always located at the edited entry. A separate test rewrote one entry and recomputed every later hash; in every one of 300 trials the hash chain alone accepted the forged log and the on-chain roots rejected it.

5.4 What each layer catches

Figure 5.1 summarises which attacks the hash chain alone can and cannot detect, and the effect of adding the on-chain roots.

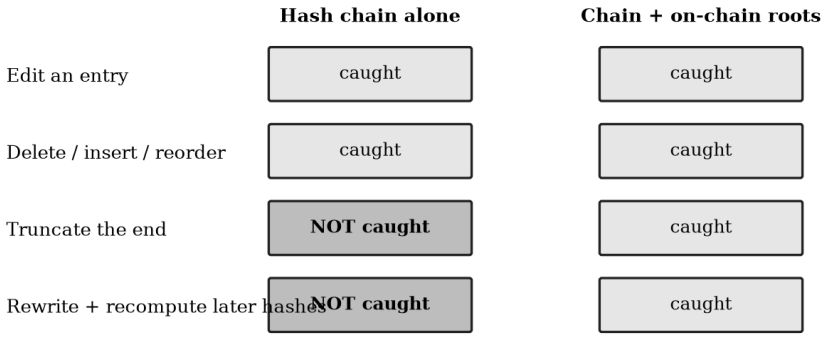


Figure 5.1 Detection by the hash chain alone and with on-chain roots

Truncating the end of the log leaves a valid, shorter chain, and a consistent rewrite leaves a valid chain, so both are invisible to the chain alone. Both are caught once batch roots are on the blockchain.

5.5 Agreement between Python and Solidity

Merkle trees of 1, 2, 3, 5, 8, 13 and 100 entries were built in Python, with an inclusion proof for every entry. Each proof was verified by the contract on the local chain, and each verification returned true. Wrong entry hashes, flipped proof sides and proofs against the wrong batch were rejected.

5.6 Run with a real local model

The local model qwen3:8b completed the task in the demonstration (Section 4.8). The log it produced passed hash-chain verification, was anchored, and then showed the same detection behaviour as the scripted runs under every attack.

5.7 Discussion

The results show that the detection logic is correct for the attacks tested and that the Python and Solidity implementations agree. They do not yet show how the system behaves at scale. The gas figures come from a single run and are not a study. The model used is small and the task simple, which is acceptable because the audit layer does not depend on model quality, but it means no claim is made about agent performance.

Two limits stand out. After a consistent rewrite the verifier can locate the altered batch but not the altered entry. And entries written after the latest anchor are not protected until the next anchor, a delay that Phase II will quantify against anchoring cost.

CHAPTER 6
CONCLUSIONS AND WORK SCHEDULE FOR PHASE II

6.1 Conclusions

A tamper-evident audit trail for LLM agent actions was designed, implemented and tested. Hash chaining detects edits, deletions, insertions and reordering of the log. Anchoring Merkle roots on a blockchain additionally detects truncation and a consistently rewritten log, at a cost of tens of thousands of gas per batch and with only hashes stored on-chain. All seven attack types were detected in all 2,100 randomised trials, untouched logs were never flagged, and Python-generated proofs verified inside the contract. A real local language model produced a log that anchored and verified end to end. The system proves that a log was not changed after anchoring. It does not prove that the logged events were true.

6.2 Work schedule for Phase II

Table 6.1 Phase II work plan

Month	**Work**	**Output**
1	Experiment on gas against batch size (sizes 1 to 1000); verify all references	Gas table and plot
2	Measure logging and anchoring overhead per agent step; verification scaling from 100 to 100,000 entries	Overhead and scaling results
3	Build the dashboard showing verified and tampered entries; optional signed entries	Demonstration application
4	Complete evaluation write-up, thesis and viva preparation	Phase II thesis

6.3 Risks

The main risk is that the measured overhead of anchoring is larger than expected for small batch sizes. That would not invalidate the work, since a measured trade-off is itself a result. A second risk is dependence on locally

installed tools; versions are therefore recorded in Table 4.1. Optional extensions, such as a public test network run, will be taken up only if the core measurements are finished on time.

APPENDIX 1
SMART CONTRACT SOURCE

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.20;

/// @title AuditAnchor
/// @notice Stores Merkle roots of agent log batches. Only hashes are stored, never raw
logs.
/// Hashing matches the Python side: leaf = sha256(0x00 || entryHash), node = sha256(0x0
1 || left || right).
contract AuditAnchor {
    struct Batch {
        bytes32 root;
        uint64 firstIndex;
        uint64 count;
        uint64 timestamp;
    }

    mapping(address => Batch[]) private batches;

    event BatchAnchored(address indexed agent, uint256 indexed batchId, bytes32 root, ui
nt64 firstIndex, uint64 count);

    /// @notice Anchor the next batch for the calling agent. Batches must be contiguous.
    function anchor(bytes32 root, uint64 count) external returns (uint256 batchId) {
        require(count > 0, "empty batch");
        Batch[] storage own = batches[msg.sender];
        uint64 firstIndex = own.length == 0 ? 0 : own[own.length - 1].firstIndex + own[o
wn.length - 1].count;
        own.push(Batch(root, firstIndex, count, uint64(block.timestamp)));
        batchId = own.length - 1;
        emit BatchAnchored(msg.sender, batchId, root, firstIndex, count);
    }

    function batchCount(address agent) external view returns (uint256) {
        return batches[agent].length;
    }

    function getBatch(address agent, uint256 batchId) external view returns (Batch memor
y) {
        require(batchId < batches[agent].length, "no such batch");
        return batches[agent][batchId];
    }

    /// @notice Check on-chain that an entry hash belongs to an anchored batch.
    /// @param sides For each proof step, true if the sibling is on the right.
    function verifyInclusion(
        address agent,
        uint256 batchId,
        bytes32 entryHash,
        bytes32[] calldata siblings,
        bool[] calldata sides
    ) external view returns (bool) {
        require(siblings.length == sides.length, "length mismatch");
        require(batchId < batches[agent].length, "no such batch");
        bytes32 current = sha256(abi.encodePacked(bytes1(0x00), entryHash));
```

```
        for (uint256 i = 0; i < siblings.length; i++) {
            current = sides[i]
                ? sha256(abi.encodePacked(bytes1(0x01), current, siblings[i]))
                : sha256(abi.encodePacked(bytes1(0x01), siblings[i], current));
        }
        return current == batches[agent][batchId].root;
    }
}
```

APPENDIX 2
MERKLE TREE MODULE

```
"""Merkle tree over entry hashes, with inclusion proofs.

Leaves are hashed as sha256(0x00 || leaf), inner nodes as sha256(0x01 || left || right).
An unpaired node is carried up unchanged, so no leaf is ever duplicated.
"""
from __future__ import annotations

import hashlib
from typing import Sequence

# A proof step is (side, sibling_hash_hex) where side is "L" or "R": where the sibling s
its.
ProofStep = tuple[str, str]


def _leaf(entry_hash_hex: str) -> bytes:
    return hashlib.sha256(b"\x00" + bytes.fromhex(entry_hash_hex)).digest()


def _node(left: bytes, right: bytes) -> bytes:
    return hashlib.sha256(b"\x01" + left + right).digest()


def _levels(entry_hashes: Sequence[str]) -> list[list[bytes]]:
    if not entry_hashes:
        raise ValueError("cannot build a Merkle tree from zero entries")
    levels = [[_leaf(h) for h in entry_hashes]]
    while len(levels[-1]) > 1:
        current = levels[-1]
        nxt = [_node(current[i], current[i + 1]) for i in range(0, len(current) - 1, 2)]
        if len(current) % 2:
            nxt.append(current[-1])
        levels.append(nxt)
    return levels


def merkle_root(entry_hashes: Sequence[str]) -> str:
    return _levels(entry_hashes)[-1][0].hex()


def merkle_proof(entry_hashes: Sequence[str], index: int) -> tuple[ProofStep, ...]:
    if not 0 <= index < len(entry_hashes):
        raise IndexError("entry index out of range")
    steps: list[ProofStep] = []
    position = index
    for level in _levels(entry_hashes)[:-1]:
        sibling = position ^ 1
        if sibling < len(level):
            steps.append(("R" if sibling > position else "L", level[sibling].hex()))
        position //= 2
    return tuple(steps)


def verify_proof(entry_hash: str, proof: Sequence[ProofStep], root: str) -> bool:
```

```
    current = _leaf(entry_hash)
    for side, sibling_hex in proof:
        sibling = bytes.fromhex(sibling_hex)
        current = _node(current, sibling) if side == "R" else _node(sibling, current)
    return current.hex() == root
```

APPENDIX 3
HASH-CHAIN MODULE

```
"""Hash-chained log entries: each entry commits to the one before it."""
from __future__ import annotations

from dataclasses import dataclass
from typing import Any, Mapping, Sequence

from core.hashing import GENESIS_HASH, canonical_json, sha256_hex


@dataclass(frozen=True)
class LogEntry:
    index: int
    timestamp: float
    payload: Mapping[str, Any]
    prev_hash: str
    entry_hash: str


@dataclass(frozen=True)
class ChainVerification:
    ok: bool
    first_bad_index: int | None = None
    reason: str = ""


def compute_entry_hash(index: int, timestamp: float, payload: Mapping[str, Any], prev_ha
sh: str) -> str:
    body = {"index": index, "timestamp": timestamp, "payload": payload, "prev_hash": pre
v_hash}
    return sha256_hex(canonical_json(body))


def make_entry(
    entries: Sequence[LogEntry],
    payload: Mapping[str, Any],
    timestamp: float,
) -> LogEntry:
    """Build the next entry for a chain (does not mutate `entries`)."""
    index = len(entries)
    prev_hash = entries[-1].entry_hash if entries else GENESIS_HASH
    return LogEntry(index, timestamp, payload, prev_hash, compute_entry_hash(index, time
stamp, payload, prev_hash))


def append_entry(entries: Sequence[LogEntry], payload: Mapping[str, Any], timestamp: flo
at) -> tuple[LogEntry, ...]:
    return (*entries, make_entry(entries, payload, timestamp))


def verify_chain(entries: Sequence[LogEntry]) -> ChainVerification:
    """Check indices, links and hashes. Reports the first entry that does not hold up.""
"
    prev_hash = GENESIS_HASH
    for position, entry in enumerate(entries):
```

```
        if entry.index != position:
            return ChainVerification(False, position, "index out of sequence (entry inse
rted, removed or reordered)")
        if entry.prev_hash != prev_hash:
            return ChainVerification(False, position, "prev_hash does not match the prev
ious entry")
        if entry.entry_hash != compute_entry_hash(entry.index, entry.timestamp, entry.pa
yload, entry.prev_hash):
            return ChainVerification(False, position, "entry content does not match its
hash")
        prev_hash = entry.entry_hash
    return ChainVerification(True)
```

APPENDIX 4
DEMONSTRATION OUTPUT

```
agent answered: The results are:
- 12*(3+4) = 84
- The word count in "auditable agents leave a trail" is 5.
- The capital of Japan is Tokyo.
12 audit entries saved to research/experiments/agent_logs/ollama_run.jsonl
anchored batch 0: entries 0..4, root ef116e522257…, gas 93100
anchored batch 1: entries 5..9, root 506981b1cf67…, gas 79518
anchored batch 2: entries 10..11, root 9c03f8fd0a9f…, gas 79518

verify untouched log: INTACT
verify after editing entry 3: TAMPERING DETECTED
   - hash chain broken at entry 3: entry content does not match its hash
   first suspicious entry: 3
verify after consistent rewrite (only the chain can catch this): TAMPERING DETECTED
   - batches not matching on-chain roots: [0, 1, 2]
   - hash chain is self-consistent, so only batch-level location is possible: entries 0.
.4
   first suspicious entry: 0
verify after deleting the last 2 entries: TAMPERING DETECTED
   - batches not matching on-chain roots: [2]
   - hash chain is self-consistent, so only batch-level location is possible: entries 10
..11
   - log is shorter than what was anchored (entries removed from the end)
   first suspicious entry: 10
verify after restoring the original: INTACT
```

APPENDIX 5
LIST OF PUBLICATIONS

No publications yet. An IEEE conference abstract is in preparation.

REFERENCES

1. Anthropic (2024), 'Building effective agents', Anthropic Engineering. [VERIFY publication date]

2. Crosby S.A. and Wallach D.S. (2009), 'Efficient data structures for tamper-evident logging', Proceedings of the 18th USENIX Security Symposium. [VERIFY pages]

3. Haber S. and Stornetta W.S. (1991), 'How to time-stamp a digital document', Journal of Cryptology, Vol. 3, No. 2, pp. 99-111. [VERIFY]

4. Laurie B., Langley A. and Kasper E. (2013), 'Certificate Transparency', RFC 6962, Internet Engineering Task Force (obsoleted by RFC 9162). [VERIFY]

5. Laurie B. et al. (2021), 'Certificate Transparency Version 2.0', RFC 9162, Internet Engineering Task Force. [VERIFY authors and year]

6. Merkle R.C. (1988), 'A digital signature based on a conventional encryption function', Advances in Cryptology, CRYPTO '87, Lecture Notes in Computer Science, Vol. 293, pp. 369-378. [VERIFY]

7. Nakamoto S. (2008), 'Bitcoin: A peer-to-peer electronic cash system'.

8. National Institute of Standards and Technology (2015), 'Secure Hash Standard (SHS)', FIPS PUB 180-4. [VERIFY year]

9. National Institute of Standards and Technology (2023), 'Artificial Intelligence Risk Management Framework (AI RMF 1.0)'. [VERIFY]

10. Schneier B. and Kelsey J. (1999), 'Secure audit logs to support computer forensics', ACM Transactions on Information and System Security, Vol. 2, No. 2, pp. 159-176. [VERIFY]

11. Wood G., 'Ethereum: A secure decentralised generalised transaction ledger', Ethereum Yellow Paper.

12. Yao S., Zhao J., Yu D. et al. (2023), 'ReAct: Synergizing reasoning and acting in language models', International Conference on Learning Representations (arXiv:2210.03629). [VERIFY venue]